

Week 1: Project Architecture and Backend Setup

Project Architecture

The first week focused on establishing the overall backend architecture for the RESQAI disaster response application.

The system was designed using a **two-service architecture**:

- **Node.js + Express** backend for API handling, business logic, authentication and database operations.
- **Python + FastAPI** ML service for BERTweet model inference and Gemini-based analysis.

This separation allows the machine learning components to remain independent from the main application backend and makes it easier to scale the ML service separately.

Backend Structure

The Node.js backend was organized into different modules:

```
```text
```

```
backend/
```

```
 config/
```

```
 controllers/
```

```
 middleware/
```

```
 models/
```

```
 routes/
```

```
 services/
```

```
 utils/
```

```
 server.js
```

The controllers were separated according to functionality:

- `authController.js` for authentication
- `incidentController.js` for incident operations
- `adminController.js` for administrative operations
- `rescueController.js` for rescue-team operations

Middleware was also introduced for JWT verification, role-based access control and centralized error handling.

### ## Database Design

MongoDB with Mongoose ODM was selected as the database.

Three primary collections were planned:

```
```text
```

```
Users
```

Incidents
ClassificationResults

The **User** collection stores authentication and role information, while the **Incident** collection stores disaster reports and their complete lifecycle. Classification results are stored separately as ML metadata for debugging and analysis.

Outcome

By the end of Week 1, the basic project architecture, backend structure and database design were established, providing the foundation for implementing authentication, incident processing and ML integration.